Manual Técnico

(Arquitectura, código, configuración, API, seguridad y operación)

Cuida Tu Agua: Plataforma web y móvil de monitoreo del consumo de agua con control remoto de la válvula

Juan Esteban Ome Esquivel

Juan Manuel Gómez

Diego Alejandro Alvarado

Equipo de desarrollo

Servicio Nacional de Aprendizaje — SENA

Centro de formación: Centro de la Industria, la Empresa y los Servicios

Regional: Regional Huila

Programa: Tecnología en Análisis y Desarrollo de Software

Ficha: 3145556

Instructor: Carlos Julio Cadena

9 de octubre de 2026

Versión: 1.0

Control de versiones

Tabla 1

Historial de versiones del documento

Versión	Fecha	Autor(es)	Descripción del cambio
1.0	09/10/2026	Equipo Cuida Tu Agua	Primera versión. Describe la arquitectura, la estructura del código, la configuración, la API, la mensajería, la seguridad, la base de datos, las pruebas, las convenciones y la operación del sistema.

Tabla 2

Revisión y aprobación del documento

Rol	Nombre	Firma	Fecha
Elaboró	Equipo de desarrollo		
Revisó	Carlos Julio Cadena (instructor)		
Aprobó			

Tabla de contenido

Control de versiones.....

Tabla de contenido.....

1. Introducción.....

2. Arquitectura.....

3. Estructura del código.....

   3.1 Servicios .NET.....

   3.2 ms-iam.....

   3.3 Aplicación save-your-water.....

   3.4 Firmware del medidor.....

4. Configuración.....

5. API REST.....

6. Mensajería.....

   6.1 RabbitMQ.....

   6.2 MQTT.....

7. Seguridad.....

8. Base de datos y errores.....

   8.1 Base de datos.....

   8.2 Formato de errores.....

9. Pruebas.....

10. Convenciones de trabajo.....

11. Operación.....

   11.1 Limitaciones conocidas.....

Referencias.....

Para actualizar la tabla: clic derecho sobre ella → Actualizar campos → Actualizar toda la tabla.

1. Introducción

Este manual reúne la información técnica que necesita un desarrollador para entender, mantener y extender Cuida Tu Agua. Complementa el Documento de Diseño del Software, que justifica las decisiones de arquitectura, y el Manual de Instalación, que explica cómo poner en marcha el sistema. Aquí se describe cómo está organizado el código, qué configura cada servicio, qué expone su API y cómo se opera.

La información corresponde al repositorio sy-water-backend del 9 de octubre de 2026, al firmware 2.0.1 y a la aplicación save-your-water. Los valores secretos se omiten.

2. Arquitectura

El sistema tiene seis microservicios con arquitectura hexagonal. Los clientes consumen API REST con JWT; el medidor se comunica por MQTT con ms-device; los servicios se integran con eventos en RabbitMQ y con llamadas HTTP internas, y cada uno es dueño de su schema en la base de datos sy-water-db (figuras 1 y 2).

Figura 1

Componentes del sistema por capas

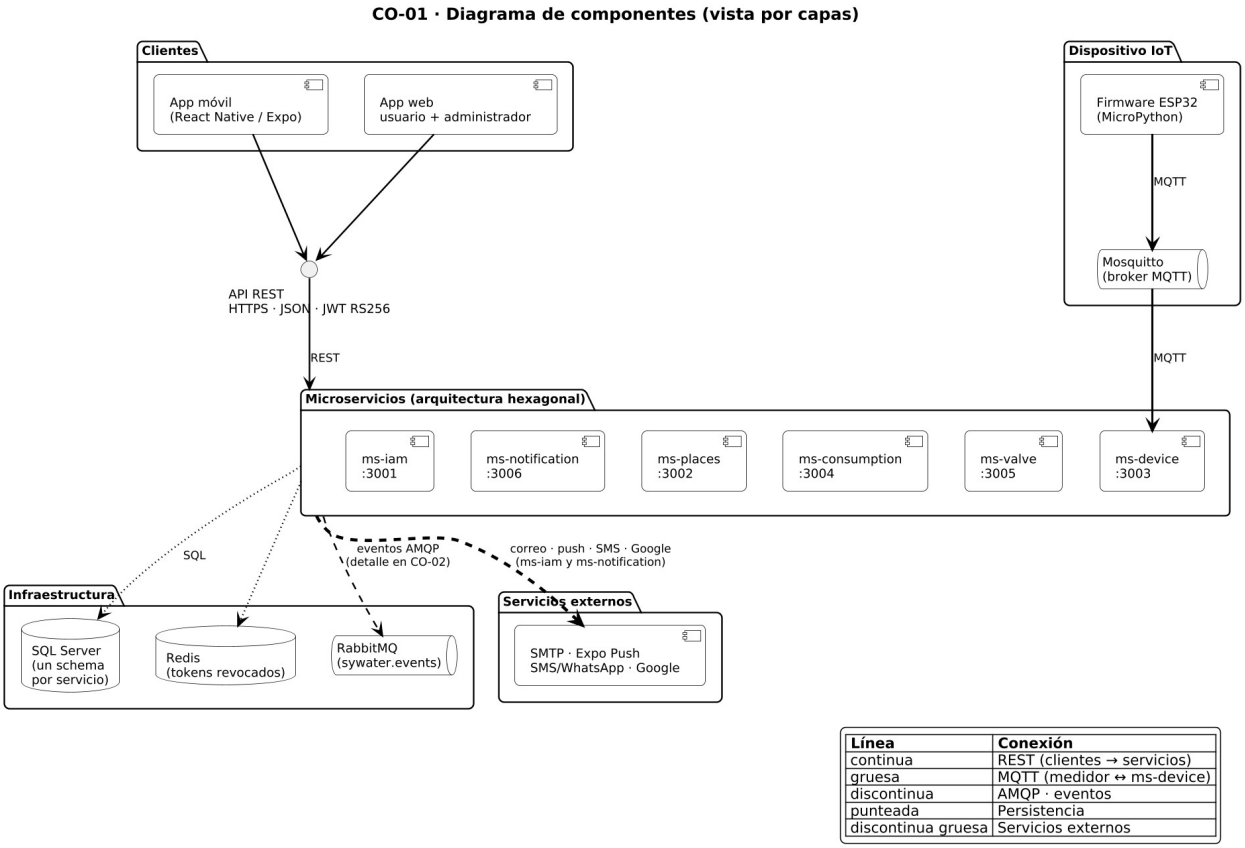


Figura 2

Integración entre microservicios

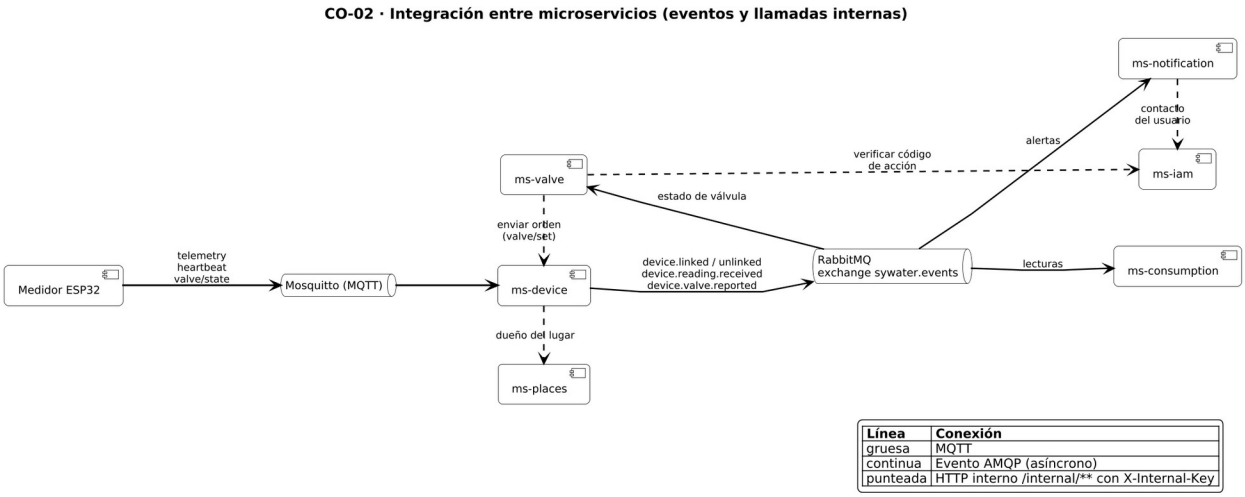


Tabla 3

Servicios del sistema

Servicio	Tecnología	Puerto	Schema	Proyecto de arranque
ms-iam	Java 21, Spring Boot 4.1.1	3001	security	mvnw spring-boot:run
ms-places	.NET 10, EF Core 10.0.12	3002	places	src/SyWater.Places.Api
ms-device	.NET 10, MQTTnet 5.2, RabbitMQ.Client 7.2.2	3003	devices	src/SyWater.Devices.Api
ms-consumption	.NET 10, RabbitMQ.Client	3004	monitoring	src/ SyWater.Consumption.Api
ms-valve	.NET 10, RabbitMQ.Client	3005	valve	src/SyWater.Valve.Api
ms-notification	.NET 10, RabbitMQ.Client, MailKit 4.18.1	3006	notification	src/ SyWater.Notifications.Api

3. Estructura del código

3.1 Servicios .NET

Cada servicio .NET es una solución (.slnx) con cuatro proyectos de producción y sus proyectos de prueba. Las dependencias apuntan siempre hacia el dominio.

```
ms-<servicio>/
├── global.json                          SDK 10.0.100
├── SyWater.<Servicio>.slnx
├── src/
│   ├── SyWater.<Servicio>.Domain/          agregados, objetos de valor,
políticas, DomainException
│   ├── SyWater.<Servicio>.Application/     Ports/In, Ports/Out, UseCases,
vistas y eventos
│   ├── SyWater.<Servicio>.Infrastructure/  Persistence (DbContext,
entidades, Ef*, mappers), clientes HTTP
│   └── SyWater.<Servicio>.Api/             Controllers, Composition,
Security, Errors, Messaging, Background, keys/
└── tests/                               Domain.Tests, Application.Tests,
Api.Tests (xUnit)
```

El cableado de dependencias está en Api/Composition/ServiceCollectionExtensions.cs. Para agregar una funcionalidad: se crea el caso de uso en Application con su puerto de entrada, se definen los puertos de salida que necesita, se implementan en Infrastructure, se registran en Composition y se expone en un controlador.

3.2 ms-iam

Tabla 4

Paquetes de ms-iam (com.sywater.ms_iam)

Paquete	Contenido
domain.model / domain.exception	User, Email, PhoneNumber, PasswordPolicy, LockoutPolicy, Role, AccountStatus; DomainException y sus subclases.
application.port.in / port.out	Interfaces de casos de uso (LoginUseCase, RegisterUserUseCase…) y puertos de salida (UserRepository, AccessTokenIssuer, TokenRevocationStore, NotificationSender…).
application.service / dto	Implementación de los casos de uso con @Transactional, registrados como @Bean en ApplicationConfig; vistas y resultados.
infrastructure	config (ApplicationConfig, SecurityConfig, KeysConfig, ClockConfig), persistence (entidades JPA, repositorios Spring Data, adaptadores y UserMapper), security (Nimbus RS256, BCrypt, Redis, InternalKeyGuard), integration (clientes HTTP), notification (SMTP), storage (avatares).
presentation	Controladores REST, DTO de petición y ApiExceptionHandler (ProblemDetail).

3.3 Aplicación save-your-water

La aplicación (Expo SDK 57, React Native 0.86, TypeScript) usa una arquitectura hexagonal híbrida. Los archivos con sufijo .web.tsx o .web.ts solo se cargan en el navegador; Android e iOS usan los originales, por lo que la versión web no modifica la móvil.

Tabla 5

Carpetas de la aplicación

Carpeta	Contenido
src/config	api.ts: URL de los servicios (EXPO_PUBLIC_*) y rutas de cada endpoint.
src/core	auth (AuthContext, TokenManager con SecureStore en móvil y localStorage en web), http (ApiClient con interceptores de renovación), di/container.ts (elige la implementación de cada puerto), notifications.
src/domain	Entidades, puertos y reglas puras por módulo (places, devices, consumption, valve, notifications, admin), sin React ni axios.
src/infrastructure	Repositorios HTTP (adaptadores), httpError.ts (códigos de error → mensajes en español), push.
src/presentation	Componentes, hooks, navegación (incluye RootNavigator.web y

Carpeta	Contenido
	AdminNavigator), pantallas y estilos del sistema de diseño.
src/web y src/admin	Diseño de escritorio (WebShell, Page, ResponsiveGrid) y panel de administración (solo web).

Sesión: en el celular los tokens se guardan en SecureStore; en la web, en localStorage. El interceptor de petición renueva el token si vence en menos de 30 segundos; el de respuesta, ante un 401, renueva una sola vez (una única renovación en curso) y repite la petición; si la renovación falla, cierra la sesión con el mensaje «Tu sesión expiró».

3.4 Firmware del medidor

Tabla 6

Archivos del firmware 2.0.1

Archivo	Función
main.py	Arranque: restaura la válvula, activa el watchdog y ejecuta el ciclo de 1 s (WiFi, reloj, lecturas, MQTT, latido, estado de válvula y botón BOOT).
config.py	Ajustes comunes: versión, tiempos, pines, calibración (7,5 pulsos por L/min) y watchdog de 60 s.
secrets.py	Datos de cada medidor (no va a Git): MQTT_HOST, MQTT_PORT, MQTT_USER, MQTT_PASSWORD, DEVICE_SERIAL, DEVICE_TOKEN, SETUP_PASSWORD.
protocol.py / link.py	Tópicos y mensajes MQTT; cliente umqtt.simple con QoS 1.
wifi.py / setup_portal.py	Conexión WiFi y portal de configuración en 192.168.4.1.
valve.py / flow.py / readings.py	Relé de la válvula con estado persistido; conteo de pulsos del sensor; búfer de lecturas de hasta una hora.
clock.py / storage.py	Hora UTC por NTP; escritura atómica de archivos JSON.

4. Configuración

Los servicios .NET leen appsettings.json y los secretos de user-secrets o de variables de entorno (con doble guion bajo, por ejemplo ConnectionStrings__Places). ms-iam lee application.yml y variables de entorno. Las claves comunes son:

Tabla 7

Claves de configuración comunes

Clave	Valor por defecto	Uso
ConnectionStrings:<Servicio>	(secreto)	Obligatoria; sin ella el servicio no arranca.
Jwt:Issuer / Jwt:KeyId / Jwt:PublicKeyPath	https://iam.cuidatuagua.local / iam-key-1 / keys/iam-public.pem	Validación de tokens RS256.
Redis:Configuration	localhost:6379	Lista de revocación (no aplica a ms-device).
RabbitMq:Host, Port, Username, Password	localhost, 5672, sywater, (secreto)	Sin Host no se publican ni consumen eventos.
Internal:ApiKey	(secreto, ≥ 24 caracteres)	Cabecera X-Internal-Key.
Services:*BaseUrl	http://localhost:300X	Llamadas entre servicios (5 s de tiempo de espera).
Cors:AllowedOrigins	[]	Orígenes permitidos para la web.

Tabla 8

Parámetros de negocio configurables o fijos

Parámetro	Valor	Dónde
Token de acceso / renovación / código de acción	1 h / 7 días / 5 min	ms-iam, iam.jwt.*
Código de verificación / recuperación; intentos; reenvío	24 h / 15 min; 5; 60 s	ms-iam, iam.codes.*
Bloqueo por intentos de inicio de sesión	5 intentos → 15 min	ms-iam, iam.lockout.*
Emparejamiento del medidor	5 fallos → 15 min; 10 intentos por usuario cada 15 min	ms-device, Devices:*
Barrido de estado del medidor	Cada 60 s; desconectado tras 10 min	ms-device, Devices:StatusSweepSeconds
Confirmación de órdenes de válvula	30 s; sin comunicación tras 3 min	ms-valve, ValvePolicy (código)
Caudal actual vigente	2 min	ms-consumption (código)
Alerta de medidor silencioso	10 min	ms-notification, Devices:OfflineAfterMinutes
Reintentos de correo	1 min, 5 min, 15 min, 1 h; fallido al quinto	ms-notification (código)
Lecturas / latido / estado de válvula del	10 s / 30 s / 60 s	Firmware, config.py

Parámetro	Valor	Dónde
medidor		

5. API REST

Todos los servicios exigen un token válido salvo las rutas marcadas como anónimas. «Interno» significa cabecera X-Internal-Key sin token de usuario. Los errores siguen el formato Problem Details (sección 8).

Tabla 9

Endpoints de ms-iam (puerto 3001)

Método	Ruta	Acceso
POST	/api/auth/register · /verify-email · /verify-email/resend · /login · /refresh	Anónimo
POST	/api/auth/password/forgot · /api/auth/password/reset	Anónimo
POST	/api/auth/logout	Usuario
GET / PATCH / DELETE	/api/users/me	Usuario
PUT	/api/users/me/password	Usuario
PUT / DELETE	/api/users/me/avatar	Usuario
POST	/api/users/me/action-codes · /api/users/me/action-codes/verify	Usuario
GET	/api/avatars/**	Anónimo
GET	/api/admin/users · /api/admin/metrics	Administrador
PUT	/api/admin/users/{userId}/block · /unblock	Administrador
GET	/internal/users/{userId}/contact	Interno
GET	/actuator/health	Anónimo

Tabla 10

Endpoints de ms-places (puerto 3002)

Método	Ruta	Acceso
GET / POST	/api/places	Usuario
GET / PUT	/api/places/{placeId}	Usuario

Método	Ruta	Acceso
/ DELETE		
PUT	/api/places/{placeId}/default	Usuario
GET	/api/places/{placeId}/tariff · /api/places/{placeId}/cost	Usuario
PUT	/api/places/{placeId}/tariff/manual · /tariff/catalog	Usuario
GET	/api/tariffs/catalog/cities/{cityId}	Usuario
GET	/api/geography/countries · /countries/{code}/subdivisions · /subdivisions/{id}/cities	Usuario
GET	/api/tips · /api/tips/favorites	Usuario
PUT / DELETE	/api/tips/{tipId}/favorite	Usuario
GET / POST	/api/admin/tips	Administrador
PUT	/api/admin/tips/{tipId} · /api/admin/tips/{tipId}/active	Administrador
GET	/internal/metrics	Interno
GET	/health	Anónimo

Tabla 11

Endpoints de ms-device (puerto 3003)

Método	Ruta	Acceso
GET / POST / DELETE	/api/places/{placeId}/device	Usuario
POST	/api/places/{placeId}/device/valve-commands	Usuario + interno (ms-valve)
DELETE	/internal/users/{userId}/devices	Interno
GET	/internal/metrics	Interno
GET	/health	Anónimo

Tabla 12

Endpoints de ms-consumption, ms-valve y ms-notification (puertos 3004 a 3006)

Método	Ruta	Acceso
GET	/api/places/{placeId}/consumption?period=day|week|month&tz=America/Bogota	Usuario

Método	Ruta	Acceso
GET	/api/places/{placeId}/valve	Usuario
POST	/api/places/{placeId}/valve/close (con código) · /valve/open	Usuario
GET	/api/places/{placeId}/valve/commands/{commandId} · /valve/commands?from&to	Usuario
GET	/api/notifications · /api/notifications/unread-count	Usuario
POST	/api/notifications/{id}/read · /api/notifications/read-all	Usuario
GET / PUT	/api/notification-preferences · /api/notification-preferences/{severity}	Usuario
POST / DELETE	/api/push-tokens	Usuario
GET	/health (los tres servicios)	Anónimo

En el entorno de desarrollo cada servicio .NET publica su especificación OpenAPI en /openapi/v1.json. Los archivos .http de cada servicio contienen ejemplos de todas las peticiones.

6. Mensajería

6.1 RabbitMQ

ms-device publica en el exchange de tipo topic sywater.events un sobre con id, type, occurredAt y data; la clave de enrutamiento es el tipo del evento. Cada cola tiene como exchange de mensajes muertos sywater.dlx, que entrega a la cola sywater.dead-letters.

Tabla 13

Eventos de integración

Evento	Cuándo se publica	Consumidores
device.linked	Se vincula un medidor a un lugar	ms-valve, ms-notification
device.unlinked	Se desvincula un medidor	ms-valve, ms-notification
device.reading.received	Llega una lectura válida del medidor	ms-consumption, ms-notification
device.valve.reported	El medidor reporta el estado de la válvula	ms-valve, ms-notification

Un mensaje mal formado o que viola una regla de dominio se rechaza sin reencolar; un error transitorio se reintenta hasta cinco veces y luego va a la cola de mensajes muertos. Los

consumidores son idempotentes: procesar dos veces el mismo mensaje no duplica lecturas ni notificaciones.

6.2 MQTT

Tabla 14

Tópicos MQTT

Dirección	Tópico	Mensaje	Frecuencia
Medidor → servidor	sywater/devices/{serial}/heartbeat	{token, fw}	30 s
Medidor → servidor	sywater/devices/{serial}/telemetry	{token, ts, lpm, vol, total}	10 s, con búfer sin red
Medidor → servidor	sywater/devices/{serial}/valve/state	{token, state, commandId}	60 s y tras cada orden
Servidor → medidor	sywater/devices/{serial}/valve/set	{commandId, action}	Bajo demanda

Mosquitto no admite conexiones anónimas. El usuario ms-devices lee los tópicos de los medidores y escribe las órdenes; el usuario sywater-device (compartido por todos los medidores) solo escribe, y cada medidor solo puede leer las órdenes de su propio serial (pattern read sywater/devices/%c/valve/set, con el serial como identificador de cliente). ms-device valida además el token de cada mensaje contra su hash en la base de datos.

7. Seguridad

- **Autenticación:** ms-iam emite un token de acceso JWT RS256 de 1 hora (con identificador jti y roles USER o ADMIN) y un token de renovación opaco de 7 días que rota en cada uso. Los demás servicios validan solo con la llave pública.
- **Revocación:** al cerrar sesión, bloquear o eliminar una cuenta, ms-iam registra el token en Redis; ms-places, ms-consumption, ms-valve y ms-notification lo consultan y responden 401 si Redis no está disponible. ms-device todavía no consulta Redis (acción correctiva ACC-09).
- **Contraseñas y secretos:** contraseñas con BCrypt; códigos, tokens y credenciales del medidor guardados solo como hash SHA-256 y comparados en tiempo constante.

- **Llamadas internas:** cabecera X-Internal-Key con una clave compartida de al menos 24 caracteres; sin clave, las rutas /internal/** responden 403.
- **Autorización:** las rutas /api/admin/** exigen el rol ADMIN; las operaciones sobre un lugar verifican que el usuario sea su dueño.
- **Limitación de intentos:** inicio de sesión (5 intentos → 15 minutos), emparejamiento del medidor (5 fallos → 15 minutos) y vinculación por usuario (10 intentos cada 15 minutos, con cabecera Retry-After).
- **Acciones de riesgo:** cerrar la válvula exige un código de 6 dígitos enviado al correo, de un solo uso y válido 5 minutos.
- **Pendiente para producción:** MQTT con TLS en el puerto 8883 y HTTPS delante de los servicios.

8. Base de datos y errores

8.1 Base de datos

La base sy-water-db en SQL Server tiene un schema por servicio. Cada schema tiene un usuario de aplicación (<x>_app, con permisos mínimos) y uno de migración (<x>_migrator). Los servicios no ejecutan migraciones al arrancar: ms-iam valida el esquema (ddl-auto: validate) y los servicios .NET no llaman a Migrate(). Las migraciones están en los repositorios ms-*-db con Liquibase.

```
ms-<x>-db/
├── compose.yml, liquibase.properties, .env.example
├── sqlserver-init/00-init.sql     schema, usuarios y permisos (se ejecuta
como sa)
├── changelog/db.changelog-master.yaml
├── 01-ddl/  02-dml/  03-dcl/  04-tcl/  05-rollbacks/
└── 06-releases/vX.Y/              cambios posteriores (nunca se edita una
release aplicada)
```

Cada changeset usa el formato SQL de Liquibase con su instrucción --rollback. Para volver a una versión: liquibase rollback <etiqueta>. El diccionario de datos completo está en el anexo B del Documento de Diseño del Software.

8.2 Formato de errores

Todos los servicios responden los errores con Problem Details (RFC 9457, application/problem+json). El campo title contiene un código estable que la aplicación traduce a un mensaje en español; la aplicación nunca muestra el texto en inglés del campo detail.

Tabla 15

Códigos de error frecuentes

Código HTTP	Ejemplos de código de dominio
400	validation.failed, auth.invalid_code, device.pairing_failed, consumption.invalid_period
401	auth.invalid_token (token vencido, revocado o mal firmado)
403	auth.admin_required, auth.internal_only, auth.account_blocked
404	place.not_found, notification.not_found
409	device.offline, device.already_linked, place.already_has_device, valve.command_in_progress, user.email_already_registered
423 / 429	auth.account_locked, device.pairing_locked / device.too_many_attempts
503	service.unavailable, mqtt.unavailable

9. Pruebas

Tabla 16

Pruebas automatizadas

Componente	Comando	Tipo de pruebas
ms-iam	.\mvnw.cmd test	Dominio y servicios con dobles escritos a mano; contrato HTTP con @WebMvcTest
Servicios .NET	dotnet test	Dominio y casos de uso con dobles en memoria; API con WebApplicationFactory y SQLite
Aplicación	npx tsc --noEmit; npm test	Tipos, reglas de dominio y adaptadores con Jest
Firmware	Arnés en PC contra Mosquitto real	Protocolo y flujo de punta a punta del medidor

Sin hardware, el medidor se simula con ms-device/scripts/simulate-device.ps1 (latidos) o con el simulador en Python de sy-water-hardware/pc-simulator (lecturas y válvula). No deben ejecutarse a la vez el simulador y un medidor real con el mismo serial.

10. Convenciones de trabajo

- **Ramas:** main y dev; una rama por historia o funcionalidad creada desde dev (por ejemplo, feature/hu-029-fuga), con pull request pequeño hacia dev y revisión cruzada.
- **Commits:** Conventional Commits en inglés, por ejemplo feat(devices): ..., test(valve): ..., docs: ...; cada commit compila y deja las pruebas en verde.
- **Código:** comentarios en inglés; textos para el usuario en español, en segunda persona y diciendo qué hacer.
- **Base de datos:** identificadores UUID, fechas en UTC, borrado lógico y bitácoras que solo admiten inserción.
- **Definición de terminado:** pruebas en verde, endpoints nuevos en el archivo .http, mensajes de error traducidos en la aplicación, cambios de base de datos como release con rollback y etiqueta, ningún secreto en Git y captura de cada pantalla nueva.

11. Operación

- **Salud:** /health en los servicios .NET (ms-device también revisa MQTT: Degraded sin broker) y /actuator/health en ms-iam.
- **Registros:** todos los servicios escriben en la consola; Mosquitto, en la salida del contenedor (docker logs sy_water_mosquitto).
- **Revisión periódica:** cola sywater.dead-letters en el panel de RabbitMQ (http://localhost:15672) y correos con estado fallido en notification.email_outbox.
- **Respaldos:** no hay respaldos automáticos; se recomienda respaldar sy-water-db a diario, junto con los volúmenes de Mosquitto y RabbitMQ y la carpeta de avatares de ms-iam (acción preventiva APR-06).

11.1 Limitaciones conocidas

- No hay Dockerfile para los servicios; se ejecutan con dotnet run y mvnw. Un despliegue en servidor requiere crear las imágenes o publicar con dotnet publish y java -jar.
- ms-notification no trae el archivo keys/iam-public.pem; hay que copiarlo antes de arrancar.

- ms-device no consulta la lista de revocación en Redis.
- La aplicación espera el servicio de consejos ms-recommendations (puerto 3007), mientras que el repositorio revisado tiene los consejos en ms-places (/api/tips). Las métricas del panel también usan rutas /summary que difieren de /api/admin/metrics. Ambas configuraciones deben unificarse.
- Los eventos de ms-device se publican directamente en RabbitMQ, sin outbox: si el broker no está disponible se pierden los de ese intervalo.

Referencias

Equipo Cuida Tu Agua. (2026a). *Guías de implementación del proyecto Cuida Tu Agua* [Documentos internos del proyecto].

Equipo Cuida Tu Agua. (2026b). *sy-water-backend* [Código fuente de los microservicios, versión del 9 de octubre de 2026].

Equipo Cuida Tu Agua. (2026c, 9 de octubre). *Documento de diseño del software: Cuida Tu Agua* (Versión 1.0) [Documento interno].

Nottingham, M., Wilde, E., y Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force.